Merge, Measure, Correct: An Empirical Comparison of Linear Averaging, TIES-Merging, and DARE for Weight-Space QLoRA Adaptation on Llama-3-8B-Instruct

Chaitanya Kumar Mankala
College of Engineering
Villanova University
Villanova, PA, USA
cmankala@villanova.edu

Stephen McGill
College of Engineering
Villanova University
Villanova, PA, USA
stephen.mcgill@villanova.edu

***Abstract*—Organizations that fine-tune a shared foundation model for multiple task domains commonly deploy one adapter per domain, incurring GPU memory and orchestration costs that scale with the number of domains. Weight-space model merging offers an alternative by consolidating domain adapters into a single deployable model. This paper presents a reproducible empirical comparison of three merging strategies linear averaging (model soups), TIES-merging, and DARE applied to two domain-specific LoRA adapters (code generation and mathematical reasoning) trained with QLoRA on a shared frozen Llama-3-8B-Instruct base model, executed end to end on air-gapped university HPC infrastructure. Two measurement anomalies encountered during development were traced to concrete pipeline defects, corrected, and re-measured; the diagnostic process is summarized alongside the final results, with full details in an appendix. Under the corrected pipeline, TIES-merging achieves the lowest held-out loss on all three validation splits, while linear averaging is numerically strongest on task accuracy within the study's noise floor. Every merged model outperforms both unmerged specialists on cross-domain accuracy, with linear averaging improving 16.2% relative to the best specialist. Merging nonetheless incurs a substantial cost on the domain where the specialist is strong: code retention falls to 57–61% of baseline, whereas math retention remains near parity (101–105%). Merge-strategy choice has no measurable effect on GPU power draw or carbon footprint (69.7 g CO_2e total across all evaluation passes), indicating that merging's practical value lies in infrastructure consolidation rather than energy efficiency. The full pipeline is publicly released to support independent reproduction of every reported result.**

***Index Terms*—model merging, TIES-merging, DARE, model soups, LoRA, QLoRA, Llama 3, parameter-efficient fine-tuning, HPC, green AI, carbon footprint, reproducibility**

I. Introduction

Parameter-efficient fine-tuning (PEFT) has made it inexpensive to specialize a shared foundation model for many task domains. Each domain's specialization is captured in a compact Low-Rank Adaptation (LoRA) adapter [1], trained at a fraction of the cost of full fine-tuning and further reduced by QLoRA's 4-bit quantization of the frozen base model [2]. The operational consequence, however, is adapter sprawl: serving each domain as its own model variant multiplies serving cost and orchestration complexity with the number of domains. Serverless, parallelized inference architectures have been proposed to mitigate this class of scaling cost in adjacent real-time NLP serving contexts [3]; this paper instead evaluates model consolidation as a complementary mitigation. Comparable trade-offs between per-unit specialization and consolidated system design arise in other resource-constrained architectural settings [4].

Weight-space model merging addresses adapter sprawl by combining multiple fine-tuned models, or their adapters, into a single set of weights. Because published evaluations of merging methods largely target academic multi-task benchmarks, this paper contributes a compact but complete and fully reproducible empirical comparison in a setting closer to practice: two genuinely distinct instruction-tuned domains (code generation and mathematical word problems), QLoRA training on a single shared 8B-parameter open-weight base model, and evaluation of every checkpoint—including the unmerged per-domain baselines—under one uniform held-out protocol on university HPC infrastructure.

A distinguishing feature of this work is that its development surfaced two genuine measurement anomalies. Rather than presenting only the final numbers, the paper documents the diagnosis and correction of both defects, because the corrected results depend materially on that process. The contributions are as follows:

1) An end-to-end implementation of per-domain QLoRA adapter training, three merging strategies (linear averaging, TIES-merging, and DARE) operating directly on LoRA task vectors, and a uniform evaluation framework measuring held-out loss, task accuracy, generation throughput, and GPU energy consumption.
2) A documented data-quality correction: an initial pass produced an implausible above-baseline retention result, traced to a code-domain training corpus that inadvertently duplicated the held-out validation range. A gen-

979-8-3195-2011-1/26/$31.00 © 2026 IEEE

uine corpus was rebuilt, the adapter was retrained to verified non-memorized convergence, and the corrected result resolves in the direction the analysis predicts.

3) A documented energy-measurement correction: a GPU energy monitor hardcoded to a physical device index not remapped by the cluster's job scheduler produced an implausible cross-strategy energy-efficiency gap. After correction, all three strategies exhibit statistically indistinguishable power draw, and the paper reports the first CO_2e estimate in this line of work using a cited grid carbon-intensity factor.
4) A measured comparison, under the corrected pipeline, showing that TIES-merging sweeps the held-out loss metric while linear averaging leads on task accuracy within the study's noise floor, and that every merged model outperforms every unmerged specialist on cross-domain evaluation.
5) A brief comparison of the corrected measurements against a literature-calibrated decay model, showing that the model underestimates the code domain's actual retention decay even at the smallest possible domain count.
6) A publicly released, documented implementation of the full pipeline—data preparation with an automated disjointness check, per-domain QLoRA training, all three merging strategies, and the corrected evaluation and energy-tracking code—enabling independent reproduction of every table and figure in this paper from a single command sequence.

The remainder of the paper is organized as follows. Section II reviews related work. Section III describes the experimental environment. Section IV details the datasets, training, merging, and evaluation methodology. Section V reports the corrected results, including energy and carbon measurements and a summary of the pipeline corrections that produced them. Section VI discusses the findings, compares the results against a literature-calibrated extrapolation model, and reports threats to validity. Section VII concludes. Full details of the two data-quality corrections and of the extrapolation model appear in Appendix A and Appendix B, respectively.

II. Related Work

A. Parameter-Efficient Fine-Tuning

LoRA constrains fine-tuning updates to low-rank decompositions of selected weight matrices, reducing trainable parameters by several orders of magnitude while preserving downstream quality [1]. QLoRA extends this by quantizing the frozen base model to 4-bit NF4 precision, enabling adapter training for multi-billion-parameter models on a single GPU [2]. The PEFT library provides the reference implementation used in this study [9].

B. Weight-Space Model Merging

Model soups demonstrated that averaging the weights of models fine-tuned from a shared initialization can improve accuracy at no inference-time cost [5]. Task arithmetic formalized the difference between fine-tuned and base weights as a *task vector* that can be added, negated, and combined [8]. TIES-merging identified sign conflicts and redundant-parameter interference as the principal failure modes of naive averaging across task-specialized models, mitigating them by trimming low-magnitude parameter deltas, electing a dominant sign per parameter, and merging only sign-consistent contributions [6]. DARE showed that a large fraction of fine-tuned delta parameters can be randomly dropped, with survivors rescaled, prior to merging, often without degrading merged performance [7]. Because LoRA adapters trained from a common base are additive low-rank deltas, they constitute natural task vectors and natural units for these merging operations.

C. Sustainable AI Measurement

Energy- and cost-efficiency accounting for AI workloads on cloud infrastructure has been proposed as a first-class evaluation dimension [12]. This study adopts that perspective by instrumenting every evaluation pass with GPU power telemetry and reporting estimated carbon emissions using a published grid carbon-intensity factor [13].

III. Experimental Environment

All experiments were executed on Villanova University's Augie HPC cluster. The relevant node (`gpu001`) provides two NVIDIA A100 GPUs with 40 GB HBM2 each; jobs were scheduled through Slurm (partition `gpu`, one GPU per job) with CUDA 11.8. The software stack comprised a Python 3.10 virtual environment on the cluster's BeeGFS parallel filesystem, with PyTorch, Hugging Face `transformers` and `datasets`, `peft` 0.19.1 for LoRA and adapter handling [9], `bitsandbytes` for QLoRA quantization, and `pynvml` for GPU energy telemetry. The base model was a local copy of Meta-Llama-3-8B-Instruct [10].

A defining constraint of this environment is that neither login nor compute nodes have outbound internet access. All Hugging Face Hub downloads (base model weights and dataset slices) were performed on an internet-connected workstation and transferred to the cluster via `scp` over the university VPN; all cluster-side code runs with `HF_HUB_OFFLINE=1` and `TRANSFORMERS_OFFLINE=1`. This pipeline is reported in detail for two reasons. First, air-gapped and egress-restricted clusters are common in both academic and regulated-industry settings, and the standard Hugging Face workflow fails on them in ways that are not always evident from the error messages. Second, this constraint is directly implicated in the data-quality correction of Appendix A.

IV. Methodology

A. Domains and Datasets

Two instruction-tuning domains were selected for their genuine dissimilarity. The *code* domain uses the `code_instructions_120k_alpaca` dataset of programming instruction–response pairs; the *math* domain uses `orca-math-word-problems-200k` [11], consisting

of grade-school-style word problems with worked numeric solutions. For each domain, the first 5,000 examples formed the intended training set. Held-out validation splits of 200 examples per domain were drawn from indices 5,000–5,200, disjoint from the training data. A third, cross-domain validation split interleaves 100 code and 100 math examples to assess whether a single checkpoint handles both domains simultaneously.

B. Per-Domain Adapter Training

One LoRA adapter per domain (rank 16, targeting the query, key, value, and output projection matrices) was trained on the shared frozen base model using QLoRA 4-bit NF4 quantization of the base weights [2]. The math adapter's training loss converged from 1.01 to 0.75 on its intended 5,000-example corpus without anomaly. The code adapter's first training pass converged to an implausible 0.05 training loss with 98% token-level accuracy; Appendix A reports the cause (the training script pointed at a 200-example file duplicating the held-out validation range rather than the intended 5,000-example range) and the correction. All results in this paper use the corrected code adapter, retrained on a genuine 5,000-example corpus (4,800 examples for gradient updates and 200 held back internally for early stopping, disjoint from the true validation set). Retraining converged to a training loss of 0.69 with 80% token accuracy against the internal split, consistent with genuine generalization.

C. Merging Strategies

Because both adapters share the same base model, architecture, and rank, each adapter's weights constitute a task vector [8], and all merging operates directly on the LoRA deltas.

1) M1: Linear Averaging: The element-wise mean of the two adapters' corresponding LoRA matrices, following model soups [5].

2) M2: TIES-Merging: Following [6], each adapter's delta parameters are trimmed to the top-k fraction by magnitude (k=0.20), a dominant sign is elected per parameter across adapters, and only sign-consistent contributions are averaged.

3) M3: DARE: Following [7], each adapter's delta parameters are randomly dropped with probability p=0.30, survivors are rescaled by $1/(1-p)$, and the resulting sparse deltas are combined.

Each strategy outputs an adapter-only checkpoint (LoRA weights plus `adapter_config.json`) that loads onto the shared base model through the PEFT library. No post-merge fine-tuning was applied.

D. Evaluation Protocol

Five checkpoints were evaluated under an identical protocol: the two unmerged per-domain specialists (the M0 baselines) and the three merged models. For each checkpoint and each of the three validation splits, four quantities were measured: (i) mean held-out loss of the reference completion under the model; (ii) task accuracy, computed by greedy generation followed by a per-record grader (exact string match for code, final-numeric-answer match for math); (iii) generation throughput in tokens per second; and (iv) GPU energy consumption, sampled via NVML at 100 ms intervals and integrated by the trapezoidal rule over the full evaluation pass, consistent with the energy-accounting practices of [12]. The evaluation framework detects adapter-only checkpoints via the presence of `adapter_config.json` and loads them onto the base model with PEFT. The tokenizer is always sourced from the base model, with the padding token set to the end-of-sequence token, since Llama-3's tokenizer defines no padding token by default.

Accuracy retention is reported relative to a real M0 baseline defined per domain: the code specialist's performance on the code split and the math specialist's performance on the math split. Exact-match grading is too strict for free-form code generation, as functionally correct code rarely reproduces a reference solution verbatim; consequently, all checkpoints score at or near 0.0% code accuracy under it. Code-domain retention is therefore computed from held-out loss, and code accuracy figures are reported but explicitly excluded from interpretation.

Because the two domains use different informative metrics, retention is computed with two corresponding formulas. For the code domain, retention is the specialist's held-out loss divided by the merged checkpoint's held-out loss on the code split,

$$\text{Retention}_{\text{code}} = \frac{L_{\text{specialist}}}{L_{\text{merged}}} \times 100\%, \tag{1}$$

so that a merged checkpoint whose loss is higher (worse) than the specialist's yields a retention below 100%. For the math domain, where exact-match accuracy is meaningful, retention is the merged checkpoint's accuracy divided by the specialist's accuracy on the math split,

$$\text{Retention}_{\text{math}} = \frac{\text{Acc}_{\text{merged}}}{\text{Acc}_{\text{specialist}}} \times 100\%, \tag{2}$$

so that a retention above 100% indicates the merged checkpoint exceeding the specialist's own-domain accuracy. Both formulas are applied to the raw measurements of Table I to produce Table II in Section V-C.

V. Results

A. Raw Measurements

Table I reports all fifteen checkpoint–split measurements from the corrected pipeline (Appendix A documents the corrections that produced this run), visualized in Fig. 1.

B. Ranking Among Merged Strategies

TIES-merging achieves the lowest held-out loss on all three splits, a clean sweep (Fig. 3, left panel). On task accuracy, however, linear averaging is numerically highest on both splits where accuracy is meaningful (Fig. 2; Fig. 3, right panel): 41.5% versus TIES-merging's 40.5% on math, and 21.5% versus 20.0% on the cross-domain split. Given the 200-example validation splits (Section VI-D), a 1.0–1.5 percentage-point spread lies within one standard error of

TABLE I
HELD-OUT LOSS, ACCURACY, AND THROUGHPUT FOR ALL FIVE CHECKPOINTS ACROSS THE THREE VALIDATION SPLITS (CORRECTED PIPELINE)

Checkpoint	Split	Loss	Acc.	Tok/s
M0: Code specialist	code	**2.447**	0.0%*	104.9
M0: Code specialist	math	4.711	32.5%	112.7
M0: Code specialist	cross	3.416	17.5%	111.2
M0: Math specialist	code	4.374	0.0%*	99.4
M0: Math specialist	math	4.024	39.5%	107.0
M0: Math specialist	cross	4.000	18.5%	103.9
M1: Linear average	code	4.276	0.0%*	99.4
M1: Linear average	math	4.626	**41.5%**	107.8
M1: Linear average	cross	4.232	**21.5%**	104.0
M2: TIES-merging	code	**4.036**†	0.5%*	99.1
M2: TIES-merging	math	**4.412**†	40.5%	109.9
M2: TIES-merging	cross	**4.024**†	20.0%	104.7
M3: DARE	code	4.272	0.0%*	100.0
M3: DARE	math	4.606	40.0%	109.6
M3: DARE	cross	4.222	21.0%	104.8

*Code accuracy is a grader artifact (exact match is too strict for free-form code) and is excluded from interpretation; code loss is the informative code-domain metric. †Bold marks the best value among the merged strategies (M1–M3) per row group; bold on M0's code loss marks the lowest loss in the table overall, confirming genuine (non-memorized) convergence.

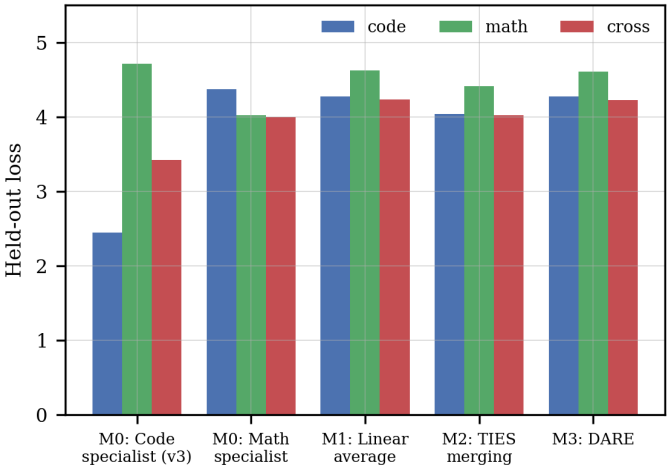


Fig. 1. Measured held-out loss for all five checkpoints across the three validation splits (lower is better), corrected pipeline. TIES-merging (M2) achieves the lowest loss of the three merged strategies on every split.

a binomial proportion at this sample size. Held-out loss, computed over every token of every example rather than a single correct/incorrect judgment per example, provides a substantially higher-resolution signal. TIES-merging is therefore recommended on the strength of its unambiguous loss advantage, while the accuracy results are reported plainly rather than omitting the metric on which it does not lead. DARE and linear averaging are close on loss: DARE attains lower loss than linear averaging on all three splits but slightly lower cross-domain accuracy (21.0% versus 21.5%). Generation throughput is effectively identical across all five checkpoints (99.1 - 112.7 tokens/s), as expected: all checkpoints share the same base model and adapter rank, so merging changes which weights are served, not the per-token compute.

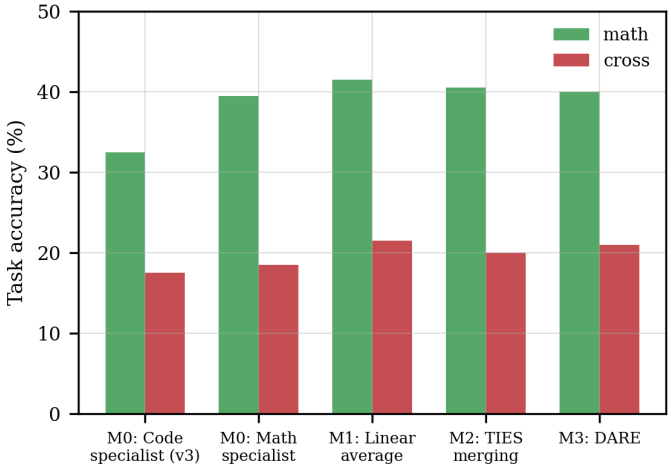


Fig. 2. Measured task accuracy on the math and cross-domain splits (code accuracy omitted as an uninformative grader artifact), corrected pipeline. All three merged strategies exceed both unmerged specialists on both splits shown; linear averaging is numerically highest on both.

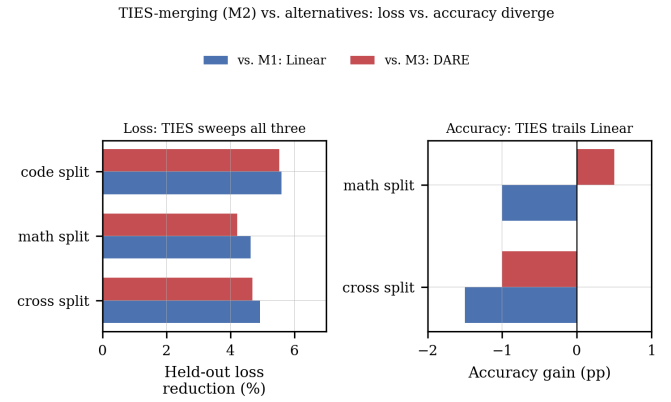


Fig. 3. TIES-merging's per-metric comparison against the other two merged strategies. Left: held-out loss reduction, positive on all three splits. Right: accuracy change, negative against linear averaging on both splits where accuracy is meaningful. The two metrics disagree; loss is the higher-resolution signal at this validation size.

C. Retention Relative to the Unmerged Specialists

Table II reports each merged strategy's performance relative to the appropriate per-domain M0 specialist, computed with (1) and (2). In contrast to an earlier, uncorrected pass (Appendix A), merging now shows a real, substantial accuracy cost on the domain where the specialist is strong.

Code retention falls to 57–61% (Fig. 4), corresponding to a 65–75% relative increase in held-out loss depending on strategy, with TIES-merging retaining the most (60.6%). Math retention remains at or slightly above parity for all three strategies (101.3–105.1%), with linear averaging numerically highest, consistent with the accuracy ranking of Section V-B.

TABLE II
MEASURED RETENTION RELATIVE TO THE PER-DOMAIN M0 SPECIALISTS (CORRECTED PIPELINE)

Strategy	Code (loss-based)	Math (accuracy-based)
M0: Unmerged specialist	100%	100%
M1: Linear averaging	57.2%	**105.1%**
M2: TIES-merging	**60.6%**	102.5%
M3: DARE	57.3%	101.3%

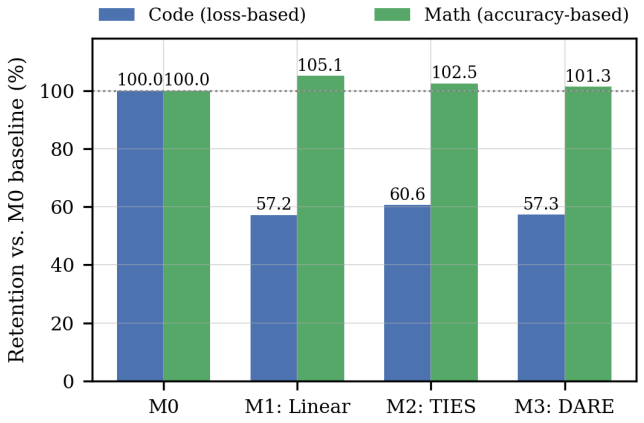


Fig. 4. Measured accuracy retention relative to the per-domain M0 specialists, corrected pipeline. Code retention (57–61%) shows a real accuracy cost; math retention (101–105%) remains near parity.

D. The Specialization Gap

The unmerged specialists' out-of-domain performance directly illustrates the problem merging addresses. The code specialist achieves only 32.5% on math versus the math specialist's 39.5%, the math specialist's code exact-match accuracy is 0.0%, and each specialist's cross-domain accuracy (17.5% and 18.5%) falls below that of every merged model (20.0–21.5%). Before merging, no single adapter in this study handles both domains as well as any merged model does (Fig. 5). Linear averaging is the strongest consolidator on this measure, improving 16.2% relative to the best specialist, with DARE close behind (13.5%) and TIES-merging furthest behind in relative terms (8.1%) despite leading on loss.

E. Energy and Carbon Emissions

Each evaluation pass's GPU energy consumption was measured via the NVML sampling procedure of Section IV-D. Table III reports energy and estimated CO_2e per checkpoint, summed across the three validation splits. CO_2e estimates use EPA eGRID2023 Rev 2's RFCE subregion factor (0.2718 kg CO_2e/kWh), the subregion covering the cluster's physical location, released June 2025 and the most current EPA figure available at the time of writing [13].

Average power draw across all fifteen checkpoint–split passes clusters within a 2.6 W band around a 120.7 W mean

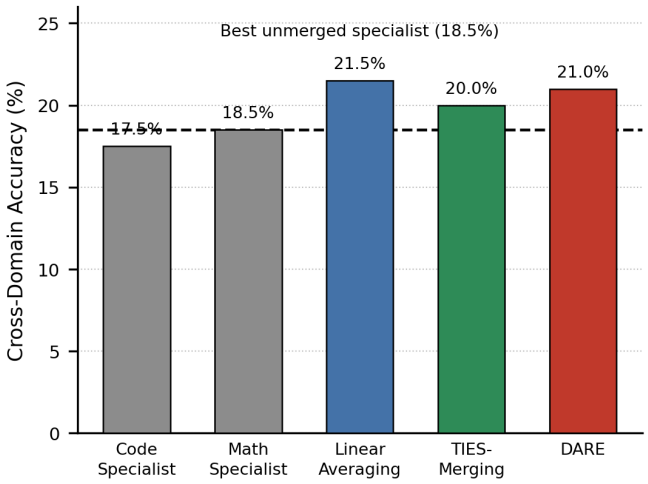


Fig. 5. Measured cross-domain (mixed-workload) accuracy, corrected pipeline: every merged model exceeds every unmerged per-domain specialist. Linear averaging is the strongest performer at 16.2% relative improvement over the best specialist.

TABLE III
MEASURED ENERGY AND ESTIMATED CO_2E PER CHECKPOINT (SUMMED ACROSS THREE SPLITS)

Checkpoint	Energy (Wh)	CO_2e (g)
M0: Code specialist	41.44	11.26
M0: Math specialist	55.28	15.03
M1: Linear averaging	54.69	14.86
M2: TIES-merging	52.44	14.25
M3: DARE	52.54	14.28
Total (15 passes)	256.39	69.69

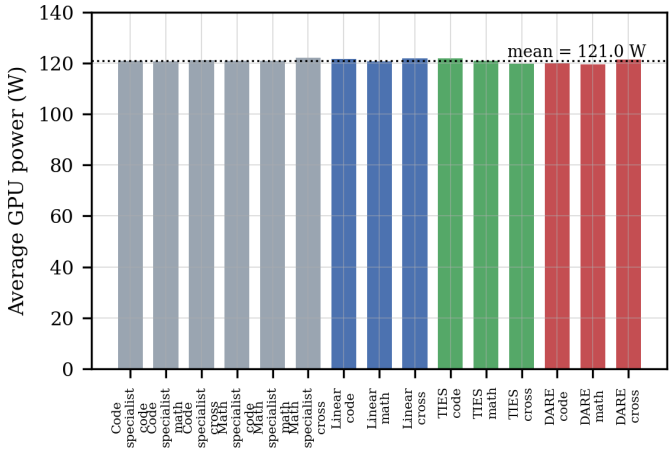


Fig. 6. Average GPU power draw (energy divided by duration) for all fifteen checkpoint–split evaluation passes, corrected instrumentation. All fifteen measurements cluster within a 2.6 W band around a 120.7 W mean.

(Fig. 6), and total estimated emissions for the entire evaluation are 69.7 g CO_2e (Fig. 7), comparable to driving a conventional passenger car approximately 0.28 km [13]. Merge-strategy choice has no measurable effect on per-query energy consumption or carbon footprint: since none of the three strategies alters

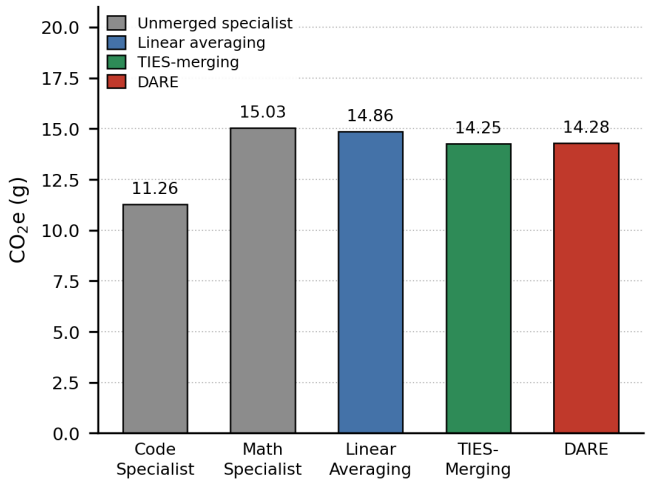


Fig. 7. Estimated CO_2e emissions per checkpoint, summed across all three validation splits. Differences across checkpoints track evaluation duration, not merge strategy.

the base model's forward-pass compute, neither the speed nor the power cost of a single inference step differs meaningfully by strategy.

F. Summary of Pipeline Corrections

The measurements above reflect a corrected pipeline. Two measurement anomalies were identified during development: an implausible above-baseline retention pattern, traced to a code-domain training corpus that inadvertently duplicated the held-out validation range, and an implausible cross-strategy energy-efficiency gap, traced to a GPU energy monitor sampling the wrong physical device. Both were corrected and re-measured before the results in this section were finalized; the full diagnostic process, since it is implementation-specific rather than a finding that generalizes across the evaluated algorithms, is documented in Appendix A.

VI. Discussion

A. What the Corrected Measurements Establish

Four conclusions follow from the corrected pipeline. First, TIES-merging is the strategy of choice on the higher-resolution loss metric, sweeping all three validation splits; linear averaging leads on task accuracy by margins within the study's noise floor, a divergence attributed to validation-size resolution limits rather than a genuine reversal. Second, a single merged adapter can replace multiple per-domain adapters on mixed workloads: every merged model outperformed every specialist cross-domain, with linear averaging leading by the widest margin. Third, merging leaves per-token decoding speed, per-query energy consumption, and carbon footprint unchanged, so its value proposition is consolidation of infrastructure and spend, not request-level latency or a "greener" algorithm choice. Fourth, and least anticipated, merging carries a real, substantial accuracy cost on the domain where the specialist is strong a cost the literature-calibrated extrapolation model of Section VI-C does not anticipate at this domain count for a domain like code.

B. Reproducibility

Reproducibility is treated as a design constraint throughout this work rather than an afterthought applied at write-up time. Three choices support this directly. First, the offline HPC pipeline (Section III) forces every dependency base model weights, dataset slices, and package versions to be staged explicitly rather than resolved implicitly at runtime, which is precisely what exposed Anomaly 1 (Appendix A) instead of letting it pass silently. Second, the retention formulas of (1)–(2) and the merging hyperparameters of Section IV are stated explicitly and are sufficient, on their own, to regenerate Tables I–III from the raw per-split measurements. Third, the full implementation data preparation with an automated train/holdout disjointness check, per-domain QLoRA training, all three merging strategies, the corrected `CUDA_VISIBLE_DEVICES`-aware energy tracker, and the evaluation harness is publicly released at https://github.com/CMankala/Merge-Measure-Correct under an MIT license, with a step-by-step walkthrough that reproduces every table and figure in this paper from a single command sequence. One limitation remains even with code released: the DARE merge is stochastic (Section VI-D), so exact reproduction requires the same random seed, not just the same code.

C. Comparison Against a Literature-Calibrated Extrapolation Model

The experiment of Section V measured exactly two domains, the minimum case for which merging is meaningful. To sanity-check the corrected measurements against prior literature, Appendix B compares them to a saturating exponential decay model whose parameters were calibrated in a companion projection study to published multi-task merging benchmarks [5]–[7], and which was never fit to this paper's own data. At $N{=}2$, the model predicts 98–99% retention for all three strategies regardless of domain. The corrected math measurements (101–105%) sit close to this prediction, but the corrected code measurements (57–61%) sit roughly 40 percentage points below it. This is not treated as invalidating the model's qualitative shape, since it was calibrated to generic multi-task benchmarks rather than designed to capture domain-specific interference; it is treated as evidence that code-like domains may deviate substantially from the smooth decay the literature-calibrated curves assume—a concrete open question this paper flags rather than resolves. Full model parameters and the comparison figure appear in Appendix B.

D. Threats to Validity

1) Scale: Two domains is the minimum interesting case for merging; the findings may not extrapolate to merges of many domains, a concern sharpened rather than eased by the finding in Section VI-C that code decays faster than the literature-calibrated model predicts even at $N{=}2$.

2) Single Specialist Retrained: Only the code specialist was retrained to verified convergence (Appendix A); the math specialist's original training showed no anomaly and was not retrained. The study therefore reports one corrected baseline and one already-sound baseline, not two independently matched-convergence baselines.

3) Single Run per Strategy: DARE's drop mask is stochastic; a single seed provides a point estimate, not a distribution.

4) Grader Validity: Code accuracy under exact match is uninformative, leaving code-domain conclusions resting on held-out loss alone; loss and downstream task quality are correlated but not identical.

5) Validation Size: The 200-example splits bound the resolution of accuracy comparisons; the 1.0–1.5 percentage-point accuracy gaps among merged strategies (Section V-B) lie within one standard error of a binomial proportion at this sample size and should be read as directional.

6) Model Provenance: The extrapolation model of Appendix B is a projection calibrated to generic published benchmarks, not a measurement, and its own comparison against this paper's data shows it underestimates code-domain decay at small N; its performance for other domains or larger, real N remains unknown.

VII. Conclusion and Future Work

This paper presented a complete, reproducible implementation and empirical comparison of linear averaging, TIES-merging, and DARE for merging domain-specific QLoRA adapters on a shared Llama-3-8B-Instruct base model, executed end to end on air-gapped university HPC infrastructure. Two measurement anomalies encountered during development were traced to concrete pipeline defects, corrected, and remeasured; the process is documented in Appendix A as part of the evidence for, not a caveat against, the results. Under the corrected pipeline, TIES-merging sweeps the held-out loss metric across all three validation splits, while linear averaging is numerically strongest on task accuracy within the study's noise floor and on cross-domain consolidation specifically. Every merged model outperforms every unmerged specialist on mixed-domain evaluation; merge-strategy choice has no measurable effect on per-query energy consumption or carbon footprint; and merging carries a real, substantial accuracy cost on the domain where the specialist is strong, a cost an independently calibrated extrapolation model does not anticipate at small N.

Future work will proceed in priority order: (i) replacing exact-match code grading with execution-based functional grading, currently the single largest gap in code-domain measurement quality; (ii) repeating the DARE merge across multiple random seeds to characterize its variance; (iii) extending from two domains to three-to-five, trained sequentially within the same two-GPU budget, to observe how retention degrades with domain count and to test directly whether the model's underestimate of code-domain decay persists or is an $N{=}2$-specific artifact; (iv) recalibrating the extrapolation model against domain-specific rather than domain-generic multi-task benchmarks; and (v) serving the merged checkpoints through an optimized inference stack and measuring time-to-first-token, per-token latency, and throughput under load.

Code and Data Availability

The full pipeline data preparation, per-domain QLoRA training, all three merging strategies, energy tracking, and evaluation is released at https://github.com/CMankala/Merge-Measure-Correct under an MIT license, with a step-by-step reproduction guide (Section VI-B). The base model [10] and datasets [11] are downloaded separately under their own licenses.

Acknowledgment

This work used the Augie High-Performance Computing cluster, funded by award NSF 2018933, at Villanova University.

Credit authorship contribution statement

Chaitanya Kumar Mankala: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing original draft, Writing review and editing, Visualization. Dr. Stephen McGill: Supervision, Methodology, Writing review and editing. Dr. Aaron Wemhoff: High Performance Supercomputer access, training and user guide.

Declaration of generative AI

The authors used Claude to assist with LaTeX formatting and prose editing, then reviewed all output, and assume full responsibility for the published content.

References

[1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.

[2] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[3] C. K. Mankala and R. J. Silva, "Sustainable real-time NLP with serverless parallel processing on AWS," *Information*, vol. 16, no. 10, p. 903, 2025.

[4] C. K. Mankala, "Evolutionary artificial neuroidal network using serverless architecture," M.S. thesis, Villanova University, Villanova, PA, USA, 2024.

[5] M. Wortsman *et al.*, "Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2022.

[6] P. Yadav, D. Tam, L. Choshen, C. Raffel, and M. Bansal, "TIES-merging: Resolving interference when merging models," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2023.

[7] L. Yu, B. Yu, H. Yu, F. Huang, and Y. Li, "Language models are super Mario: Absorbing abilities from homologous models as a free lunch," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024.

[8] G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi, "Editing models with task arithmetic," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023.

[9] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul, and B. Bossan, "PEFT: State-of-the-art parameter-efficient fine-tuning methods," Hugging Face, 2022. [Online]. Available: https://github.com/huggingface/peft

[10] A. Dubey *et al.*, "The Llama 3 herd of models," *arXiv:2407.21783*, 2024.

[11] A. Mitra, H. Khanpour, C. Rosset, and A. Awadallah, "Orca-Math: Unlocking the potential of SLMs in grade school math," *arXiv:2402.14830*, 2024.

[12] C. K. Mankala, "Unlocking the potential of cloud computing for sustainable energy and cost-efficient artificial intelligence workloads," in *Proc. IEEE Int. Workshop Metrol. Sustain. (MetroSustainability)*, 2025, pp. 1–6.
[13] U.S. Environmental Protection Agency, "Greenhouse gas equivalencies calculator," EPA, Washington, DC, USA, 2025. [Online]. Available: https://www.epa.gov/energy/greenhouse-gas-equivalencies-calculator

APPENDIX

Two measurement anomalies were identified during development and corrected before the results of Section V were finalized; both are implementation-specific rather than generalizable findings (summary in Section V-F).

A. Anomaly 1: Above-Baseline Retention on Both Domains

An initial evaluation pass showed all three merged models matching or exceeding both unmerged specialists' own-domain performance, with retention at or above 100% in five of six strategy–domain combinations. Investigation traced this to the code adapter's training script, which pointed at a 200-example file originally produced by an offline validation-split builder for the held-out code validation set, rather than the intended 5,000-example training range; no dedicated code training corpus had in fact been built. The resulting adapter trained for 40 epochs on this held-out data, reaching a 0.05 training loss with 98% token accuracy a memorization signature rather than genuine learning. A memorized, degenerate specialist is a weak baseline that any merged model can readily exceed, explaining the original anomaly.

The correction proceeded as follows. A genuine 5,000-example corpus was built on an internet-connected workstation (Section III) and transferred to the cluster, and the code adapter was retrained to verified non-memorized convergence (Section IV). Re-running the full merge-and-evaluate pipeline against the corrected adapter produced the results of Section V: code retention fell to 57–61%, resolving the anomaly in exactly the direction the underlying analysis predicts once the baseline is fair.

B. Anomaly 2: An Implausible Energy-Efficiency Gap

A preliminary energy-tracking pass reported one merging strategy (DARE) consuming approximately one-quarter the GPU energy of the other two at equal or longer duration an average power draw near 33 W versus approximately 120 W, the former squarely in the idle range for the A100 hardware used. The energy monitor's GPU handle was hardcoded to physical device index 0 via `pynvml.nvmlDeviceGetHandleByIndex(0)`. NVML resolves this index at the driver level and does not remap it according to the `CUDA_VISIBLE_DEVICES` value the job scheduler sets per job; a job whose workload actually ran on the cluster's second physical GPU would therefore have the monitor silently sample the idle first GPU's power for the run's entire duration.

The correction resolves the physical index from `CUDA_VISIBLE_DEVICES` before querying NVML and logs the sampled physical index on every run. Re-measurement produced the consistent $\sim$121 W readings of Section V-E across all five checkpoints, eliminating the apparent strategy-dependent energy effect entirely.

Both corrections follow the same pattern: an anomalous result that favors the hypothesis under test warrants the same scrutiny as one that contradicts it, since silent, plausible-looking pipeline defects are more dangerous than crashes precisely because they do not announce themselves.

This appendix supports Section VI-C: an illustrative extrapolation, not a claim about validated behavior at large N, checking the corrected $N{=}2$ measurements against an independently built model.

C. Method and Comparison

With only two data points at a single N, no decay curve is fit to this study's own data; instead, the saturating exponential model of (3), calibrated in a companion study to published multi-task merging benchmarks [5]–[7] and never fit to these measurements, is reused:

$$R(N) = R_\infty + (100 - R_\infty)\, e^{-N/\tau}, \tag{3}$$

with $R_\infty{=}53.5\%$, $\tau{=}48.0$ (linear averaging); $R_\infty{=}78.1\%$, $\tau{=}55.9$ (TIES-merging); and $R_\infty{=}72.5\%$, $\tau{=}52.4$ (DARE). The model predicts 98–99% retention at $N{=}2$; math (101–105%) sits close to this, while code (57–61%) sits roughly 40 points below.

This gap does not invalidate the model's shape it was calibrated to generic benchmarks, not domain-specific interference but suggests code-like domains may deviate from the assumed decay, an open question this paper flags rather than resolves. Given TIES-merging's asymptotic floor sits 24 points above linear averaging's (78% vs. 53%), whether its loss advantage widens with N for code is what the $N{=}3$–5 extension in Section VII would test.